

PRACTICAL OPERATING GUIDE

Agentic Data Ops

User manual

Install the capability. Understand the evidence.
Resolve ambiguity. Reassess with confidence.

DATA MATURITY AGENT 0.1.1

Comprehensive installation and usage reference
Prepared 19 September 2026



Local-first Python CLI | Excel, CSV and TSV
Includes worked examples, command reference and troubleshooting

How to use this manual

Start with installation and the synthetic quickstart. The middle chapters cover real assessments and the human feedback loop; the final chapters are the operating reference. Click a contents entry to jump to it.

01 Understand the capability	3
02 Install on Linux or macOS	4
03 Install on Windows	5
04 Your first successful assessment	6
05 Prepare your own data	7
06 Inspect and profile	8
07 Analyze and interpret the outcome	9
08 Navigate the output bundle	10
09 Read measurements without overclaiming	11
10 Create a configuration	12
11 Control size and sampling	13
12 Define explicit quality rules	14
13 Quality-rule recipes	15
14 Evaluate mission fitness	16
15 Supply governance metadata	17
16 Answer SME questions	18
17 Record decisions and accepted risk	19
18 Reassess after changes	20
19 Create and compare named baselines	21
20 Use mock or local model reasoning	22
21 Configure remote inference and roles	23
22 Security and responsible handling	24
23 Interpret maturity and AI readiness	25
24 Operate and maintain the capability	26
25 Troubleshoot installation and input	27
26 Troubleshoot assessments and models	28
27 Command reference	29
28 Glossary, sources and release scope	30

Conventions: commands run from the repository root in an active virtual environment unless stated otherwise. Replace uppercase placeholders and use a new output folder for every run. Linux/macOS commands use a backslash for line continuation; Windows PowerShell uses a backtick.

CHAPTER 01

Understand the capability

What the MVP does, who uses it, and what its results mean

Agentic Data Ops contains **Data Maturity Agent 0.1.1**, a Python command-line application. It inspects spreadsheets and delimited files, measures data quality, identifies missing business knowledge, and produces an evidence-backed assessment plus a proposed data-product contract. Use it as an analyst to understand a new asset, as a steward to identify ownership and definition gaps, or as an engineer to compare assessments after changes. A terminal runs the application; JSON, YAML and Markdown files are the deliverables. There is no web application to start.

The assessment workflow

Stage	What happens
Inspect and measure	Discover datasets and physical fields; calculate profiles, explicit quality rules, key candidates and relationship candidates.
Interpret and assess	Keep semantic and governance questions separate from measurements. Evaluate maturity, AI readiness and explicit mission requirements.
Resolve and improve	Record attributed SME answers; propose schemas, contracts and remediation; reassess and compare against history or a named baseline.

Three knowledge states

OBSERVED means a measurement or an attributed supplied fact. An authoritative human answer is observed as human-provided knowledge, not independently proven truth. **INFERRED** means a supported hypothesis with confidence and reasoning. **UNRESOLVED** means the available evidence cannot establish the answer.

Keep the distinctions

A valid assessment, a clean file, an established baseline and fitness for a particular mission are different outcomes. The application never turns a model opinion into an authoritative definition or an automatic data repair.

Start without an LLM. Model-backed reasoning is optional and adds explanatory notes; deterministic measurement and scoring remain available without a server, GPU or cloud account.

CHAPTER 02

Install on Linux or macOS

Recommended first installation; Linux is the verified platform

You need Git, a terminal, a writable project folder, and Python **3.11-3.14** with pip and venv. The package declares Python 3.11 or newer; this manual uses the tested 3.11-3.14 range. Dependency installation needs package-index access. Assessment without an LLM does not need the internet.

```
git --version
python3 --version
git clone https://github.com/drcrael/agnostic-data-ops.git
cd agnostic-data-ops
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev]"
data-maturity --help
```

The virtual environment keeps dependencies separate from other applications. After activation, use **python** and **python -m pip** so installation and execution use the same interpreter. The help screen should list inspect, profile, analyze and resolve.

Choose the installation type

Command from the repository root	Use
<code>python -m pip install -e ".[dev]"</code>	Recommended for following this manual: runtime, test and build tools; source edits take effect immediately.
<code>python -m pip install .</code>	Regular runtime installation. Retain the clone for configuration examples, fixture generation and documentation.

Return to the application later

```
cd /path/to/agnostic-data-ops
source .venv/bin/activate
data-maturity --help
```

Replace `/path/to/agnostic-data-ops` with your actual folder. Use `deactivate` to leave the environment. On some Linux distributions, a missing `venv/ensurepip` component must be installed using your distribution package manager. macOS commands are provided for convenience; end-to-end validation was performed on Linux.

CHAPTER 03

Install on Windows

PowerShell commands and a path that does not require activation

Install Git and a Python version in the tested 3.11-3.14 range. Open PowerShell in a writable folder. The example selects Python 3.11 with the Windows Python launcher; choose another installed version in the tested range if appropriate.

```
git --version
py -3.11 --version
git clone https://github.com/drcrael/agnostic-data-ops.git
cd agnostic-data-ops
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e ".[dev]"
.\.venv\Scripts\data-maturity.exe --help
```

These commands call the environment directly, so you do not need to change PowerShell execution policy. If your environment permits activation, run the following and use the shorter commands shown elsewhere in this manual.

```
.\.venv\Scripts\Activate.ps1
data-maturity --help
```

Run the demo without a shell script

```
.\.venv\Scripts\python.exe scripts/generate_test_workbooks.py
.\.venv\Scripts\data-maturity.exe analyze `
  tests/fixtures/dirty_customers.xlsx --no-llm `
  -o output/windows-demo-01
```

A PowerShell backtick at the end of a line continues the command; do not place spaces after it. The Linux/macOS examples use backslashes for continuation instead. Alternatively put the whole command on one line.

Platform scope

Windows has not been validated end to end. The sh scripts and resource-based performance test target Unix-like environments. Use the direct Python/CLI commands for a first smoke test; investigate any platform-specific failure before relying on Windows for a production process.

CHAPTER 04

Your first successful assessment

A reproducible walkthrough with the included synthetic fixtures

Run these commands from the repository root with the environment active. The fixture generator creates synthetic Excel workbooks that are not stored as binary files in the repository.

```
python scripts/generate_test_workbooks.py
data-maturity inspect tests/fixtures/semi_structured.xlsx
data-maturity profile tests/fixtures/dirty_customers.xlsx
data-maturity analyze tests/fixtures/dirty_customers.xlsx \
  --no-llm -o output/dirty-01
```

The analyze command prints a lifecycle outcome, dataset count and output location. **WAITING_FOR_HUMAN** is expected when definitions or governance decisions remain unknown. It means the reports are available and you have specific questions to resolve.

Open these files first

File in output/dirty-01	What to do
maturity_report.md	Read the overview, measured scope, findings and prioritized remediation.
sme_questions.md	Identify the questions requiring an accountable owner or domain expert.
quality_report.json	Check measured values, explicit rule outcomes and evidence references.
assessment_history.json	Read the latest iteration, decision, reasons and next actions.

One-command demo on Linux/macOS

```
sh scripts/demo.sh
```

The script generates the fixtures and assesses dirty_customers.xlsx without an LLM into a timestamped output folder. It is an assessment demonstration, not an automated cleanup.

Repeat safely

Use a new output name such as output/dirty-02 on subsequent runs. Assessment reports refuse a nonempty destination; profile output requires a destination that does not already exist. Keep prior reports for comparisons.

CHAPTER 05

Prepare your own data

Supported formats, workbook layout and source identity

Input	Practical preparation
.xlsx / .xlsm	Prefer explicit Excel tables or rectangular regions with one header row. Leave blank rows/columns between independent tables. Hidden content may be inspected.
.csv / .tsv	Use UTF-8 text, a first-record header and the fixed comma/tab delimiter. Give columns meaningful, distinct names.
Other formats	Legacy .xls, encrypted workbooks, databases, Parquet, JSON and arbitrary delimiters are not supplied ingestion paths.

Save the source before running an assessment. Retain an unchanged copy if you intend to answer questions later: resolve checks the original file hash. The application reads the source and does not edit it.

Avoid ambiguous layouts

Multirow headers, nested visual reports, adjacent tables without separators and complex merged layouts may not be discovered correctly. Start with inspect and review each detected region and field list. If the layout is wrong, prepare a tabular export and assess that export as a separately documented artifact. There is no CLI region-selection switch.

Treat formulas and identifiers deliberately

Formulas and macros are not executed. Formula results are unavailable to profiling and contribute to missing-value counts with an explicit disclosure. To assess calculated values, create a separately identified, values-only export through your controlled source process. Keep the original and document the derivation.

Preserve leading zeros in identifiers. Excel strings and numbers are physically different; CSV uses lexical interpretation. Do not silently normalize IDs, dates, units or codes just to remove findings. Agree on the intended meaning and normalization first.

Organize source versions

Use a stable filename for successive versions of the same asset, for example incoming/customers.xlsx. Archive old versions in separate folders. Dataset and field IDs depend on structural identity; renames can look like removal and addition rather than a simple value change.

CHAPTER 06

Inspect and profile

Use the smallest command that answers your current question

Inspect: confirm the structure

```
data-maturity inspect "incoming/customer export.xlsx" \  
  --config configs/no_llm.yaml > inspection.json
```

inspect writes JSON to standard output. It reports source metadata and detected datasets, including structure and fields. Check dataset_id, worksheet/table, region boundaries, original and canonical names, confidence and structural disclosures. There is no --output option for inspect; the shell redirection above saves its output.

The command performs no LLM inference. Configuration still controls ingestion bounds. Source names that normalize to the same canonical name receive deterministic disambiguation rather than being silently merged.

Profile: inspect measurements and evidence

```
data-maturity profile incoming/customers.xlsx \  
  --config configs/no_llm.yaml -o output/profile-01
```

profile creates **profile.json**, containing dataset profiles and evidence. Without -o it prints the same payload to standard output. It forces inference off. It does not produce the full mission, governance, question, contract or longitudinal assessment bundle; use analyze for those workflows.

Output-directory difference

The profile command requires a nonexistent directory. analyze and resolve accept a new or empty directory and stage the full bundle atomically. The dedicated profile file and redirected inspection file use ordinary filesystem permissions; protect them with your working-directory access controls.

When inspection is not what you expected

Confirm the input extension and format, review hidden sheets and blank boundaries, and inspect original field names before writing rule or mission mappings. Use the stable dataset/field IDs from the result when human-readable names are ambiguous. Source preparation is preferable to interpreting the wrong region as your intended dataset.

CHAPTER 07

Analyze and interpret the outcome

A full assessment can succeed while questions or defects remain

```
data-maturity analyze incoming/customers.xlsx --no-llm \  
--config configs/default.yaml -o output/customers-01
```

analyze runs the complete assessment and writes the report bundle. --no-llm overrides llm_enabled in the selected configuration. If -o is omitted, runtime.output_directory is used; its default is output. Prefer explicit, unique output folders.

Final outcome	Meaning and next action
WAITING_FOR_HUMAN	Open blocking authoritative questions remain. Send the questions and relevant evidence to the appropriate SME; record answers with resolve.
DEGRADED	No blocking human questions, but an optional reasoning component failed. Review component_status and model configuration; deterministic results remain available.
REMEDIATION_REQUIRED	Authority gates pass, but material findings remain. Correct data or rules through an approved process, then reassess.
BASELINE_ESTABLISHED	Authority gates pass without material findings. This lifecycle state does not itself prove mission fitness or create a named baseline unless one was requested.
Fatal failure / exit 2	Invalid input, configuration, policy or output-integrity error. Correct the issue and rerun; do not treat a failed command as a usable assessment.

The human gate takes precedence over optional reasoning failure. A WAITING_FOR_HUMAN run can also contain degraded components; inspect component_status in assessment.json rather than relying only on the terminal summary.

Use exit codes correctly

Exit code **0** means a valid assessment was produced. It is not a data acceptance signal. If integrating into another process, evaluate the lifecycle decision, relevant quality rules, mission results and sample scope explicitly. Shell argument errors also typically return 2.

CHAPTER 08

Navigate the output bundle

Human reports, machine-readable records and audit artifacts

Artifact	Contents and use
assessment.json	Canonical full assessment: datasets, profiles, assertions, findings, governance, maturity, readiness and component status.
maturity_report.md	Human-readable overview, limitations, findings, questions and remediation roadmap.
sme_questions.md	Targeted questions with identifiers for structured responses.
schema.json	Observed physical schema and proposed target schema per dataset.
quality_report.json	Quality metrics and explicit rules, status, denominator, scope and evidence references.
evidence.json	Measurements and provenance objects. It is not a full source-row export.
evidence_graph.json	Nodes/edges and explicit relationships used for traceability; no graph database is required.
data_contract.yaml	Proposed contracts, candidate keys, relationships and unresolved expectations.
mission_fitness.json	Requirement-level results when mission evaluation produces results.
run_manifest.json	Hashes and reproducibility metadata for the bundle.
assessment_history.json	Iterations, decisions, resolutions, snapshots and named baselines.
assessment_delta.json	Changes when a prior assessment or named baseline is compared.

Keep the whole bundle together. Loading `assessment.json` can automatically pick up its adjacent `assessment_history.json`; a mismatch is rejected. For longitudinal work, pass the history file explicitly so baselines and prior resolutions remain available.

Markdown files open in a text editor or Markdown viewer; JSON/YAML files are also plain text. Do not edit generated history or evidence to change a result. Edit trusted configuration, submit resolutions, or correct the source and produce a new assessment.

CHAPTER 09

Read measurements without overclaiming

Scope, denominators and candidate findings

Check scope before interpreting a percentage

A profile describes either full measured data or a sample. The default retains up to 10,000 rows while scanning up to 1,000,000 rows. A scan-limited run describes a sample of the scanned prefix, not an unbiased sample of the entire file. Sampled uniqueness and containment do not establish population-wide keys or orphans.

Measurement	Interpretation
Null rate	Uses all measured rows as its denominator. Formula results that are unavailable are included as missing; consult the formula finding.
Distinctness / uniqueness	Uses non-null values. Duplicate frequency counts occurrences beyond the first. Three copies of a value contribute two duplicates.
Physical types	Values are type-sensitive: integer 1, string "1" and boolean true differ. All-null columns have no observed type.
Dates and anomalies	Ambiguous slash dates are not guessed. Statistical outliers and unusual text are candidates for review, not proof of invalid data.
Key and join candidates	Complete/unique fields and bounded two-field key combinations are candidates. Relationships use conservative same-name scalar identifiers and compatible physical types.

Separate evidence from interpretation

A measured missing ID can support an inferred expectation that the field should be required. Only an explicit not_null rule makes that requiredness a user-defined rule. Likewise, duplicate candidate identifiers do not prove a violation of an approved business key.

For a finding, inspect its state, severity, affected count/percentage and evidence_refs. For explicit rules, review PASS, FAIL or UNDETERMINED and the explanation. WARNING generally means the measurement exists but the interpretation remains a candidate.

No automatic repairs

Do not delete anomalies, fill blanks, deduplicate records or change codes solely to improve the report. Confirm business meaning and source authority first; keep transformation lineage and rerun on the resulting asset.

CHAPTER 10

Create a configuration

One YAML document defines the settings for each invocation

Start from configs/default.yaml and save your own configuration outside the tracked example files. `--config` loads one YAML document. Omitted fields use built-in defaults; it does not merge several configuration files or automatically include another YAML file. Unknown fields are rejected by the validated model.

```
llm_enabled: false
runtime:
  output_directory: output
  random_seed: 42
  verbosity: INFO
profiling:
  max_rows: 1000000
  sample_rows: 10000
  include_value_summaries: false
security:
  mode: local_only
  allow_metadata_to_remote_models: false
  allow_raw_data_to_remote_models: false
llm_context:
  include_samples: false
```

```
data-maturity analyze incoming/customers.xlsx --no-llm \
--config my-config.yaml -o output/configured-01
```

Reproducible changes

Version your trusted configuration alongside your operating procedure. Keep the source hash, seed and bounds with the output bundle. To compare runs after a rule change, preserve the `rule_id`, increment `quality_rules_version` and supply the complete updated configuration with `--prior`.

`analyze` does **not** automatically restore the prior configuration: pass `--config` again when you used custom settings. `resolve`, when `--config` is omitted, restores the configuration embedded in the prior assessment. A prior mission context is reused unless a new `--mission` file is supplied.

Configuration files are limited to 2 MB and use YAML safe loading. Credentials belong in the named environment variable, never in YAML. A `.env` file is not automatically loaded. Relative input, mission, configuration and output paths resolve from your current working directory.

CHAPTER 11

Control size and sampling

Resource settings and their default values

profiling setting	Default	Operational meaning
sample_rows	10,000	Maximum retained rows per dataset for measurement.
max_rows	1,000,000	Maximum scanned rows; exceeding it is disclosed as truncation.
max_columns	256	Maximum admitted columns; validated upper bound is 16,384.
max_file_bytes	100,000,000	Maximum input-file bytes.
max_excel_cells	2,000,000	Bound on workbook cells considered by ingestion.
max_uncompressed_bytes	250,000,000	Bound on expanded workbook content.
max_field_chars	100,000	Maximum field/value character size.
max_composite_candidates	100	Maximum candidate two-field key combinations; zero disables them.
max_relationship_pairs	5,000	Bound on relationship candidate pair evaluation.
top_values	20	Bound for local common-value summaries when enabled.
include_value_summaries	false	Opt in to common values/enums in local reports.

When you need full coverage

Use inspect to understand the source and raise both sample_rows and max_rows enough to cover the intended dataset, while retaining appropriate byte/cell bounds. Confirm the actual full/sample scope in the result. Increasing retained rows raises memory usage; Excel also loads a bounded in-memory workbook.

Other useful defaults

runtime.random_seed is 42; verbosity is INFO (also DEBUG or WARNING). llm_max_attempts is 2 (1-5 allowed) and llm_cache is false. The cache, if enabled, exists only within one gateway instance. assessment_control.max_iterations is 10; this MVP still performs only one automatic pass per explicit event.

The measured 100,000-row CSV benchmark retained a 10,000-row sample. It is not a claim that every Excel workbook or a fully retained 100,000-row assessment uses the same memory or time.

CHAPTER 12

Define explicit quality rules

Turn approved expectations into deterministic checks

Rules live inside the complete configuration under `quality_rules`. Each rule needs `rule_id`, `field`, `dimension` and `operator`. Use stable, unique IDs. `dataset` is optional; omit it only when the same field rule should apply across every selected dataset. Missing fields produce an `UNDETERMINED` rule result; a nonmatching dataset selector can skip the rule entirely.

```
llm_enabled: false
quality_rules_version: "1"
quality_rules:
  - rule_id: customer_id_required
    dataset: Customers
    field: customer_id
    dimension: completeness
    operator: not_null
    severity: critical
    origin: USER_DEFINED
  - rule_id: customer_id_unique
    dataset: Customers
    field: customer_id
    dimension: uniqueness
    operator: unique
    severity: high
    origin: USER_DEFINED
```

```
data-maturity analyze tests/fixtures/dirty_customers.xlsx \
--no-llm --config customer-rules.yaml -o output/rules-01
```

Dataset selectors match a dataset ID, worksheet name or table name. Field selectors match a field ID, source name or canonical name. Use IDs when names collide, and confirm that `quality_report.json` includes the rule IDs you expected.

Requiredness is a separate rule

`unique`, `range`, `enum`, `pattern`, `type` and `freshness` evaluate non-null values. Pair them with `not_null` when the field is mandatory. A unique result does not imply completeness. Severity can be `critical`, `high`, `medium`, `low` or `info`; origin defaults to `USER_DEFINED`.

The repository includes `examples/customer_rules.yaml` and `configs/quality_rules.yaml`. The latter applies an explicit temperature range to the synthetic anomaly fixture. Do not copy a temperature bound into real data until its units and meaning are authoritative.

CHAPTER 13

Quality-rule recipes

Operator reference and date-aware freshness

Operator	parameters	Meaning / caveat
not_null	None	Counts missing values.
unique	None	Counts duplicate non-null occurrences beyond the first.
range	min and/or max	Inclusive numeric limits; nonnumeric non-null values fail.
enum	values: [CA, NY]	Exact, type-sensitive membership in the approved list.
pattern	pattern: '[A-Z]{2}'	Full-string regex match. Match and total evaluation time are bounded.
type	type: string	Expected physical type, not an automatic conversion.
freshness	max_age_days: 7	Age against assessment time; timezone-aware timestamps required.

```
quality_rules:
- rule_id: observed_recently
  dataset: Events
  field: observed_at
  dimension: timeliness
  operator: freshness
  parameters: {max_age_days: 7}
  severity: high
  origin: MISSION_DEFINED
- rule_id: observation_time_required
  dataset: Events
  field: observed_at
  dimension: completeness
  operator: not_null
```

For example, 2026-09-19T12:00:00Z carries a timezone. A date-only value or a naive timestamp does not: the freshness result stays UNDETERMINED. Document whether the timestamp represents event, publication or processing time. The age rule alone does not reject future timestamps.

Regex patterns must be nonempty, at most 256 characters, and avoid prohibited nested repetition. A match timeout or total evaluation timeout produces UNDETERMINED rather than a pass. Write simple patterns in trusted configuration, never from untrusted workbook instructions.

Use reported physical types for type rules; commonly string, integer, float, boolean, date or datetime. CSV interpretation is lexical while Excel preserves its physical value types. Increment quality_rules_version when an approved rule definition changes.

CHAPTER 14

Evaluate mission fitness

Assess explicit requirements for an intended use

A mission file explains the intended use and maps requirements to datasets and fields. Generic data quality is not a substitute for this context. The fictional example in `examples/manufacturing_operations.yaml` checks part identity, measurement units and reference frame. It does not certify manufactured parts. A small custom mission document can look like this:

```
mission_name: Customer operations
mission_description: Prepare an accountable customer directory.
data_requirements:
  - id: all-customers-have-id
    description: Every customer record has an identifier.
    dataset: Customers
    fields: [customer_id]
    check: completeness
    threshold: 1.0
    criticality: critical
  - id: customer-owner-known
    description: An accountable owner is documented.
    dataset: Customers
    check: governance
    property: owner
    criticality: high
```

```
data-maturity analyze tests/fixtures/dirty_customers.xlsx \
--no-llm --mission customer-mission.yaml -o output/mission-01
```

Supported checks are completeness, uniqueness, semantic, governance, rule, freshness and unspecified. Completeness/uniqueness thresholds are fractions from 0 to 1. Semantic checks use an assertion property such as `coordinate_unit`; governance checks use a property such as `owner`. rule and freshness use `rule_id` to reference an explicit quality rule in `--config`.

Results are **FIT**, **PARTIALLY_FIT**, **NOT_FIT** or **UNDETERMINED** per requirement. Unknown authority or sampled-only quantitative evidence prevents a positive full-data claim. A missing mapped dataset is **NOT_FIT**. Narrative-only requirements cannot be executed and remain unresolved.

Check the mapping and evidence

Review the `field_ids` and `dataset_ids` in `mission_fitness.json`. For critical semantic acceptance, verify authority for every intended field; do not infer full field coverage from a single related assertion or the headline result. `max_age_days` in a mission document alone does not create a freshness rule.

CHAPTER 15

Supply governance metadata

Document accountability without asking a model to invent it

Governance can be supplied in trusted configuration or through attributed human resolutions. Only record statements you are authorized to supply; the tool records the supplied fact but does not verify the person against an identity service.

```
governance_metadata:
  Customers:
    owner: Customer Operations
    steward: Customer Data Steward
    authoritative_source: Approved customer master export
    source_system: Example CRM
    security_classification: Internal
    retention: Per approved retention schedule R-01
    provenance: Controlled export procedure EX-01
    lineage: CRM customer entity to Customers worksheet
    version: "1"
```

Illustrative values only. Replace these with your actual approved owners, classifications, references and retention requirements. Do not label a real dataset Internal simply because that value appears in the example.

Supported governance properties

owner, steward, authoritative_source, source_system, provenance, lineage, sensitivity, security_classification, access_constraints, retention, update_cadence, version, licensing and quality_accountability.

The lookup order is wildcard "*", worksheet name, table name, then dataset ID; later entries override earlier entries. Quote "*" in YAML if you use it. Use wildcard values only for facts that truly apply to every dataset.

Apply and verify

```
data-maturity analyze incoming/customers.xlsx --no-llm \
  --config governed-config.yaml -o output/governed-01
```

Review each governance item in assessment.json. Missing owner, authoritative_source and security_classification can create blocking questions. Other missing properties still matter even when they do not block the lifecycle decision.

Metadata is not enforcement

Writing an access constraint, classification or retention statement does not change filesystem permissions, delete expired records, configure a catalog or enforce an enterprise policy. Those controls must exist in your operating environment.

CHAPTER 16

Answer SME questions

Record human knowledge with identity, authority and provenance

1. Open `sme_questions.md` from the latest run. Choose the current question ID and read its related fields/evidence. Obtain the answer from the appropriate domain expert or accountable owner. Do not reuse an example identifier.
2. Save an `answers.yaml` file. The short format below answers one question; add more entries under `answers` for a batch. For this walkthrough, choose the governance owner question in `output/mission-01` and replace its ID below. The names are illustrative; use real approved authority for your own data.

```
answers:
  "question:REPLACE_WITH_ACTUAL_ID":
    answer: Customer Operations
    answered_by: Customer Data Steward
    authority: Accountable customer-data owner
    authoritative: true
    source_reference: Approved ownership register
```

3. Resolve against the unchanged source. Without `--config`, `resolve` restores the prior embedded configuration, including its model policy.

```
data-maturity resolve output/mission-01/assessment.json \
  answers.yaml --source tests/fixtures/dirty_customers.xlsx \
  -o output/resolved-01
```

The command uses the synthetic customer mission from Chapter 14. For a real assessment, substitute its original source file. A changed hash is rejected and requires `analyze --prior` first.

4. Review the new assertions, question status, history and delta. An authoritative response creates an OBSERVED assertion with provenance and supersession links. A non-authoritative answer remains INFERRED and leaves the authority gate open. Other unanswered questions can still yield WAITING_FOR_HUMAN.

Keep the history

Use a new output directory and keep the old bundle. Resolutions create new assertion versions rather than overwriting historical knowledge. A human answer cannot erase a deterministic quality violation; correct the data, update an explicit approved rule, or use an appropriate lifecycle decision.

CHAPTER 17

Record decisions and accepted risk

The full resolution format supports more than simple answers

```
resolutions:
- resolution_id: resolution:decision-001
  question_id: question:REPLACE_WITH_ACTUAL_ID
  resolution_type: ACCEPTED_RISK
  value: Defer this question for a restricted pilot.
  provided_by: Accountable owner
  authority: Data-product owner
  authoritative: true
  source_reference: Approved decision record DR-001
  rationale: Restricted use with a documented compensating review.
  scope: Named pilot and dataset version only.
  review_at: "2027-01-01T00:00:00Z"
```

This is a format example, not a suggested decision. Replace the identity, target, rationale, scope and review date with an actual approved record. review_at must be in the future at execution time and include a timezone. Risk acceptance does not supply a missing unit or repair incorrect data.

Resolution type	Use
SME_ANSWER / DATA_OWNER_DECISION / GOVERNANCE_DECISION	Create attributed knowledge against an existing question or assertion.
AUTHORITATIVE_REFERENCE / USER_OVERRIDE	Supply a referenced answer; authority rules still apply. Override cannot rewrite deterministic facts.
ACCEPTED_RISK	Requires authority, rationale, scope and future review date. Expiry reopens applicable questions/findings on reassessment.
ACKNOWLEDGED / REMEDIATION_PLANNED	Record a finding lifecycle decision. Acknowledgement alone does not close an authoritative question.
RULE_UPDATE	Not accepted as an executable answer. Update trusted configuration and run analyze --prior instead.

Full resolutions require resolution_id, value, provided_by and at least one current question_id, assertion_id or finding_id. Use a fresh resolution ID for a new decision. Do not target a superseded assertion or edit stored history. Run the same resolve command with the full-format YAML file.

CHAPTER 18

Reassess after changes

Carry forward history while recalculating the affected components

```
data-maturity analyze incoming/customers.xlsx --no-llm \
--config my-config.yaml \
--prior output/resolved-01/assessment_history.json \
-o output/reassessed-01
```

Use the newest history and retain the same source filename for the same logical asset. Supply your custom configuration again. A new --mission replaces the prior mission context; otherwise the prior context is retained. To explicitly clear mission requirements, pass an empty mission YAML mapping, {}.

Changed input	Expected behavior
Source bytes or profiling/sampling inputs	Remeasure as needed. Changed source bytes conservatively require confirmation of old authoritative answers.
SME answers	Reuse unchanged measurements; update assertion versions and derive affected assessment results.
Rules or governance metadata	Reingest bounded rows while reusing unchanged profiles; reevaluate rule/governance effects.
Mission, maturity or model settings	Reuse measurements and recompute applicable derived components. Model changes may rerun optional reasoning.
No changed inputs	Record the invocation and stop according to no-change behavior. There is no background loop waiting to invent new knowledge.

Read the delta and iteration record

assessment_delta.json separates quality, schema, semantic, governance, maturity, AI-readiness, mission and contract changes. Findings can be new, persistent, resolved or regressed. Read the supporting scope and evidence rather than treating every difference as deterioration.

assessment_history.json records triggers, components executed/skipped, resolutions, snapshots and decisions. Use these records to explain why a result changed. Source versions, rule versions and sampled scope can legitimately change the observed outcome.

Feedback is explicit

Each invocation handles one event. The MVP has no monitoring daemon, autonomous remediation executor or continuous scheduler. Accepted risk is reviewed during execution; the application does not wake itself when a review date arrives.

CHAPTER 19

Create and compare named baselines

An immutable reference assessment for future evaluation

A named baseline captures a reference assessment after its authority gates pass. It may still contain material findings and yield REMEDIATION_REQUIRED. It is not a certificate of perfect data or permission to use the dataset for every mission.

Create the reference

```
data-maturity analyze incoming/customers.xlsx --no-llm \  
  --config my-config.yaml \  
  --prior output/resolved-01/assessment_history.json \  
  --baseline approved-v1 -o output/baseline-01
```

If blocking questions remain, resolve them or record a legitimate accepted-risk decision first. Do not fabricate answers to make the gate pass. You can also supply --baseline to resolve when the final batch of answers should establish the reference.

Compare a later source version

```
data-maturity analyze incoming/customers.xlsx --no-llm \  
  --config my-config.yaml \  
  --prior output/baseline-01/assessment_history.json \  
  --compare-baseline approved-v1 -o output/compared-01
```

The named baseline must exist in the supplied history. Its stored snapshot hash is validated before comparison. Names cannot be reused for a different baseline. Use a new name such as approved-v2 after a later approved change.

Preserve the evidence chain

Keep history, snapshots and manifests with each reference. Editing a snapshot can invalidate baseline comparison. Ordinary JSON history is not a signed or tamper-proof audit system; use your organization's archive and approval controls when that level of assurance is needed.

Read dimensions independently

A semantic definition can improve while quality regresses; a rule can be tightened while source data stays unchanged. Baseline comparison exposes these changes separately. Proposed-contract drift is supported, but external schema-registry enforcement is not implemented.

CHAPTER 20

Use mock or local model reasoning

Optional inference without changing deterministic measurement

Exercise the model path without a model server

```
data-maturity analyze tests/fixtures/ambiguous_manufacturing.xlsx \
--config configs/test_mock_llm.yaml -o output/mock-01
```

The deterministic Mock provider is for exercising the integration. Its output is not evidence of a real model's semantic accuracy. The default/no-LLM path remains suitable for a first assessment.

Connect an existing Ollama model

Install and start Ollama using its official installation instructions, and make a suitable model available under the name you intend to use. This capability does not install servers, download weights or select a model for you. Copy configs/local.yaml and replace YOUR_INSTALLED_MODEL in each configured role.

```
llm_enabled: true
security:
  mode: local_only
llm_context:
  include_samples: false
models:
  semantic_inference:
    provider: ollama
    model: YOUR_INSTALLED_MODEL
    base_url: http://127.0.0.1:11434
    timeout: 30
    json_schema: true
```

```
data-maturity analyze incoming/customers.xlsx \
--config local-model.yaml -o output/local-model-01
```

Local endpoints must use a literal loopback address: 127.0.0.1 or [::1]. localhost and private LAN addresses are classified as remote. For an existing vLLM or compatible local server, use provider: openai_compatible and its loopback base URL, commonly ending in /v1.

A loopback endpoint does not prove the server itself avoids remote forwarding. Verify the server configuration. Real downloaded models were not executed during MVP delivery validation; transport, retry and policy behavior were tested using controlled transports.

CHAPTER 21

Configure remote inference and roles

Explicit metadata permission and secrets supplied at runtime

Use a remote service only when you are authorized to transmit the selected data to that endpoint. The following template enables metadata-only transmission; replace the endpoint and model with values for your existing service.

```
llm_enabled: true
security:
  mode: hybrid
  allow_metadata_to_remote_models: true
  allow_raw_data_to_remote_models: false
llm_context:
  include_samples: false
models:
  semantic_inference:
    provider: openai_compatible
    model: YOUR_CONFIGURED_MODEL
    base_url: https://YOUR_SERVICE/v1
    api_key_env: DATA_MATURITY_API_KEY
    timeout: 30
    json_schema: true
```

Set DATA_MATURITY_API_KEY through your shell or secret manager before invoking the CLI. Do not place its value in the configuration, source file or report. .env files are not automatically read. Avoid credentials, query strings or fragments in base_url; remote inference requires HTTPS.

Independent reasoning roles

semantic_inference, quality_reasoning, governance_reasoning, maturation_reasoning and report_generation can each have a provider/model entry. Missing roles are skipped. Explanations are attributed in analyst_notes; deterministic reports are available even when report_generation is absent.

Provider settings

Defaults: timeout 30 seconds, temperature 0, max_output_tokens 2048 and json_schema true. Set json_schema false when your compatible service supports JSON output but not the requested schema format; responses still undergo validation. Global llm_max_attempts defaults to 2 and is bounded at 5.

An unavailable model can degrade optional reasoning; it must not supply measurements or authority. A policy violation fails before transmission. To rerun deterministically, use analyze --no-llm with a new output directory.

CHAPTER 22

Security and responsible handling

Protect inputs, outputs, configuration and model traffic

Boundary	What the application does
Untrusted workbook content	Treats values and metadata as data. Model output has no tool execution interface; instructions in cells cannot change policy or deterministic scores.
Local-only inference	Rejects remote-configured inference before transmission. Loopback-only classification avoids hostname ambiguity.
Metadata-only model context	Allowlisted field names/types, aggregate counts and reference information. Excludes full rows, common values, extrema and human answers.
Optional raw samples	Disabled by default. include_samples must be enabled; remote transmission additionally needs raw-data permission. Sampling is bounded.
Transport and references	Disables redirects and environment proxies. Validates JSON and existing evidence/field references with bounded retries.
Assessment bundles	Stages reports atomically and uses owner-only filesystem permissions on Unix. Does not overwrite a nonempty destination.

Your operating responsibilities

Keep trusted configuration and answer files under controlled access. Field names, metadata, aggregate statistics, extrema and human answers in local reports may themselves be sensitive. No full DLP system, encryption-at-rest service, enterprise IAM or tamper-proof history store is included.

Do not paste secrets or unnecessary personal information into a resolution. Do not publish source files or reports merely because the application code is public. A local model server must be configured not to forward content remotely.

Scope of the verified defenses

The security suite exercised policy denial, payload minimization, prompt injection and fabricated-reference rejection. Schema validation cannot prove every model explanation correct. Keep inferred text subject to human review, especially when it affects operational interpretation.

For inspection redirection and profile.json, use a protected working directory: those paths do not use the full assessment bundle writer. Windows permissions differ from Unix and require local review.

CHAPTER 23

Interpret maturity and AI readiness

Criteria are explicit and cumulative, not model-generated scores

Maturity is reported independently across ten dimensions: structure, schema_definition, data_quality, semantic_clarity, metadata, provenance_lineage, governance, machine_accessibility, data_contracts and ai_readiness. Do not average them into an unsupported overall certification.

Level	Name	How to read it
0	UNMANAGED	The first level's criteria have not all been established.
1	STRUCTURED	First-level requirements satisfied for this dimension.
2	DEFINED	Levels 1 and 2 satisfied cumulatively.
3	GOVERNED	All requirements through level 3 satisfied.
4	OPERATIONALIZED	All requirements through level 4 satisfied.
5	AI_READY	All configured requirements through level 5 satisfied.

Each dimension includes satisfied_criteria, missing_criteria, next_level_requirements, blocking_findings and evidence_refs. Criteria differ by dimension. Inspect the actual requirements rather than interpreting the shared level name as proof of enterprise-wide maturity.

Separate readiness result

AI readiness considers accessibility, structure, semantics, quality, provenance, lineage, governance, documentation, contracts, stability, observability and intended-use suitability. It returns READY, NOT_READY or UNDETERMINED with blockers. A clean file can remain NOT_READY because authority, monitoring or mission evidence is absent.

Customize responsibly

The default model is in configs/maturity_model.yaml and packaged with the application. To customize it, put the model's version and dimensions under the top-level maturity_model key in your complete configuration. Levels must be contiguous from level_1, with nonempty requirements. Unknown criterion names never pass.

Editing the standalone example YAML does not automatically change the packaged model used by an ordinary invocation. A new criterion requires code support, not just a new label. Operational authority/contract/monitoring criteria may remain unmet in this MVP; generating a proposed contract does not establish operational adoption.

CHAPTER 24

Operate and maintain the capability

A repeatable local operating procedure

For each asset and iteration

1. Preserve the source artifact and its controlled version. **2.** Review discovery before interpreting findings. **3.** Run with an explicit configuration and unique output folder. **4.** Read scope, lifecycle and mission results. **5.** Route questions to accountable people. **6.** Record decisions, remediate upstream and reassess. **7.** Archive the complete bundle and configuration together.

Validate an installation or update

```
sh scripts/test.sh
pytest -m security
sh scripts/demo.sh
python -m build
```

These commands require the dev installation. The test script runs Ruff, mypy and coverage-enabled pytest. The build command produces a source distribution and wheel. Fixtures and tests are synthetic; they do not establish domain fitness of your real dataset.

Update intentionally

```
git status --short
git pull --ff-only
python -m pip install -e ".[dev]"
sh scripts/test.sh
```

Review and preserve local source/configuration changes before pulling. The manual targets the release tag listed on its last page; a newer repository revision may change behavior. Record the revision you actually run with `git rev-parse HEAD`. `requirements-dev.lock` captures the original development dependency set; it is not a universal cross-platform compatibility guarantee.

Storage and automation

History snapshots grow over iterations; archival and compaction are manual. Maintain backups and access controls. There is no built-in scheduler. If you wrap the CLI in an external job, choose unique output folders and inspect structured decisions rather than treating exit 0 as a quality pass.

To uninstall the package from an active environment: `python -m pip uninstall data-maturity-agent`. That does not delete sources or reports. Retain assessment records according to your actual retention policy.

CHAPTER 25

Troubleshoot installation and input

Resolve environmental and structural problems first

Symptom	Likely cause and next step
data-maturity not found	Activate the correct venv or call its executable directly. Verify <code>python -m pip show data-maturity-agent</code> . A fallback is <code>python -m data_maturity.cli --help</code> .
venv / ensurepip unavailable	Install the appropriate venv support for your interpreter through your platform's normal package-management process; then recreate the environment.
PowerShell activation blocked	Use <code>.venv\Scripts\python.exe</code> and <code>.venv\Scripts\data-maturity.exe</code> directly. The manual does not require weakening execution policy.
Source path not found	Run from the repository root for relative examples; quote paths containing spaces. Confirm the actual file extension.
Fixture workbook missing	Run <code>python scripts/generate_test_workbooks.py</code> before using the .xlsx examples.
Unsupported or unreadable workbook	Use XLSX/XLSM or UTF-8 CSV/TSV. Encrypted/corrupt workbooks and legacy .xls need a controlled export first.
Unexpected tables or headers	Inspect region boundaries, blank separators, hidden content and canonical names. Prepare a simpler tabular export if discovery is wrong.
More missing values than expected	Check <code>formula_results_unavailable</code> and scope. Unexecuted formula results contribute to null counts; they are not confirmed blank source cells.
Input-size or cell bound exceeded	Review the applicable profiling limit and available resources. Increase deliberately or split/export the source; do not assume a partial report is complete.
Output already exists	Choose a new output folder. profile requires a nonexistent destination; analyze/resolve allow a new or empty one.

Keep useful diagnostics

Capture the exact command without secrets, package/repository version, exception summary, source format and applicable configuration settings. Prefer a synthetic reproduction instead of sharing a sensitive workbook or full report.

CHAPTER 26

Troubleshoot assessments and models

Distinguish missing authority from an execution problem

Symptom	What to check
Configuration or model validation failed	Check YAML indentation, field names, enum values, duplicated rule IDs and the 2 MB limit. Compare against a minimal valid configuration; full raw validation content is suppressed.
WAITING_FOR_HUMAN persists	Inspect current blocking questions. Non-authoritative answers remain open; unrelated unanswered questions can still block. Use current IDs, not superseded targets.
resolve says source changed	Analyze the current source with --prior into a new bundle, then answer questions from that bundle using the same current source.
Rule seems absent	Check dataset selector and field mapping. A dataset mismatch can skip it; an absent field produces UNDETERMINED. Verify rule_id in quality_report.json.
Freshness is UNDETERMINED	Supply timezone-aware timestamps and an explicit timeliness freshness rule. A narrative requirement or max_age_days in mission context alone is insufficient.
local_only forbids remote inference	Use a genuine loopback server with a literal IP. localhost and LAN addresses count as remote. Use --no-llm if inference is unnecessary.
Remote metadata/raw permission denied	Review policy and intended transmission. Metadata and raw samples require separate permission. Do not loosen policy merely to silence an error.
Model connection / auth / JSON failure	Verify running server, exact model name, base URL, key environment variable and timeout. Try json_schema: false for a JSON-only compatible server.
DEGRADED or missing analyst notes	Inspect component_status and configured roles. Optional model failure can preserve valid deterministic results. Missing roles are intentionally skipped.
Named baseline not found or invalid	Supply the history containing that exact name and intact snapshot. Do not repair history by hand; restore the valid archived bundle.

Known remaining boundaries: complex Excel layouts, cross-name/composite foreign-key discovery, real-model accuracy, enterprise identity, external contract enforcement and automatic monitoring. These are not settings that a user can turn on in the shipped MVP.

CHAPTER 27

Command reference

Exact command and option availability in version 0.1.1

Command	Arguments and options
inspect	SOURCE; --config FILE; --help. Structure JSON on stdout; no --output.
profile	SOURCE; --config FILE; --output DIR / -o DIR; --help. profile.json or stdout; no inference.
analyze	SOURCE; --config FILE; --output DIR / -o DIR; --mission FILE; --no-llm; --prior FILE; --baseline NAME; --compare-baseline NAME; --help.
resolve	ASSESSMENT ANSWERS; required --source FILE; --output DIR / -o DIR; --config FILE; --baseline NAME; --help.

```
data-maturity --help
data-maturity inspect --help
data-maturity profile --help
data-maturity analyze --help
data-maturity resolve --help
```

Path and option rules

SOURCE, ASSESSMENT and ANSWERS are positional file arguments. -o and --output are aliases. --no-llm exists on analyze, not resolve; resolve inherits prior settings unless you supply --config. Mission selection is on analyze; resolve preserves the prior mission.

--prior accepts assessment.json or assessment_history.json. Full history is required for named baseline comparison. --baseline requests creation of an immutable name after gates pass; --compare-baseline selects a name already in the supplied history.

Useful shell patterns

```
data-maturity inspect incoming/customers.csv > inspection.json
data-maturity profile incoming/customers.tsv > profile.json
data-maturity analyze incoming/customers.xlsx --no-llm \
-o output/run-01
```

Shell redirection can overwrite a file and follows the shell's permissions; the application's output-directory protection does not apply to redirected stdout. On PowerShell, use one line or replace a trailing backslash with a backtick for multiline commands.

CHAPTER 28

Glossary, sources and release scope

Use this manual with the specific MVP it documents

Term	Meaning
Dataset / field	A discovered tabular region and one of its columns; each has an identifier used by rules, questions and evidence.
Evidence reference	An ID linking an assertion, rule result or finding to recorded measurement/provenance.
Assertion / finding	An assertion represents knowledge or uncertainty. A finding identifies a measured issue, candidate concern or unresolved gap.
SME / authority	Subject-matter expert and the attributed basis for accepting an answer. Identity is supplied, not enterprise-authenticated.
Proposed contract	Machine-readable target schema and expectations for review. It does not enforce a registry or runtime contract.
Iteration / baseline	An explicit assessment event; an immutable named snapshot selected as a reference for future comparisons.

Documentation basis

Prepared 19 September 2026 for Data Maturity Agent **0.1.1**, release tag **v0.1.1**. Instructions were checked against the shipped CLI, configuration models, evaluators, controller, report writer and repository documentation.

[Versioned repository and README](#)
[Executed validation report | MVP limitations](#)
[Assessment control loop | Security model](#)

Verified release scope

See `VALIDATION.md` and the release verification report for the executed test results, installation checks and platform coverage for this version. These are software verification results, not guarantees about your data or real-model semantic accuracy.

First-use completion check

You can display CLI help, generate fixtures, produce a no-LLM assessment in a new folder, identify its scope and lifecycle, locate evidence and questions, and explain which facts still require human authority. Then repeat the workflow on a controlled copy of your own asset.